



**Московский государственный технический
университет им. Н. Э. Баумана
(МГТУ им. Н. Э. Баумана)**

Факультет «Информатика и системы управления»

**Кафедра «Система обработки информации и управления»
(ИУ5)**

Курс «Парадигмы и конструкции языков программирования»

**Лабораторная работа №2
«Объектно-ориентированные возможности языка Python»**

Выполнил:
студент группы ИУ5-32Б:
Микаелян С. В.

Проверил:
преподаватель каф. ИУ5
Гапанюк Ю. Е.

Задание:

1. Необходимо создать виртуальное окружение и установить в него хотя бы один внешний пакет с использованием `pip`.
2. Необходимо разработать программу, реализующую работу с классами. Программа должна быть разработана в виде консольного приложения на языке Python 3.
3. Все файлы проекта (кроме основного файла `main.py`) должны располагаться в пакете `lab_python_oop`.
4. Каждый из нижеперечисленных классов должен располагаться в отдельном файле пакета `lab_python_oop`.
5. Абстрактный класс «Геометрическая фигура» содержит абстрактный метод для вычисления площади фигуры. Подробнее про абстрактные классы и методы Вы можете прочитать [здесь](#).
6. Класс «Цвет фигуры» содержит свойство для описания цвета геометрической фигуры. Подробнее про описание свойств Вы можете прочитать [здесь](#).
7. Класс «Прямоугольник» наследуется от класса «Геометрическая фигура». Класс должен содержать конструктор по параметрам «ширина», «высота» и «цвет». В конструкторе создается объект класса «Цвет фигуры» для хранения цвета. Класс должен переопределять метод, вычисляющий площадь фигуры.
8. Класс «Круг» создается аналогично классу «Прямоугольник», задается параметр «радиус». Для вычисления площади используется константа `math.pi` из модуля [math](#).
9. Класс «Квадрат» наследуется от класса «Прямоугольник». Класс должен содержать конструктор по длине стороны. Для классов «Прямоугольник», «Квадрат», «Круг»:
 - Определите метод `get`, который возвращает в виде строки основные параметры фигуры, ее цвет и площадь. Используйте метод `format` - <https://pyformat.info/>
 - Название фигуры («Прямоугольник», «Квадрат», «Круг») должно задаваться в виде поля данных класса и возвращаться методом класса.
10. В корневом каталоге проекта создайте файл `main.py` для тестирования Ваших классов (используйте следующую конструкцию - <https://docs.python.org/3/library/main.html>). Создайте следующие объекты и выведите о них информацию в консоль (N - номер Вашего варианта по списку группы):
 - Прямоугольник синего цвета шириной N и высотой N.
 - Круг зеленого цвета радиусом N.
 - Квадрат красного цвета со стороной N.
 - Также вызовите один из методов внешнего пакета, установленного с использованием `pip`.
11. **Дополнительное задание.** Протестируйте корректность работы Вашей программы с помощью модульного теста.

Код программы:

Файл figure.py

```
from abc import ABC, abstractmethod

class Figure(ABC):
    """
    Абстрактный класс «Геометрическая фигура»
    """
    @abstractmethod
    def square(self):
        """
        содержит абстрактный метод для вычисления площади фигуры.
        """
        pass
```

Файл color.py

```
class FigureColor:
    """
    Класс «Цвет фигуры»
    """

    def __init__(self):
        self._color = None

    @property
    def colorproperty(self):
        return self._color

    @colorproperty.setter
    def colorproperty(self, value):
        self._color = value
```

Файл rectangle.py

```
from lab_python_oop.figure import Figure
from lab_python_oop.color import FigureColor

class Rectangle(Figure):
    """
    Класс «Прямоугольник» наследуется от класса «Геометрическая фигура».
    """
    FIGURE_TYPE = "Прямоугольник"

    @classmethod
    def get_figure_type(cls):
        return cls.FIGURE_TYPE

    def __init__(self, color_param, width_param, height_param):
        """
        Класс должен содержать конструктор по параметрам «ширина», «высота» и
        «цвет». В конструкторе создается объект класса «Цвет фигуры» для хранения цвета.
        """
        self.width = width_param
        self.height = height_param
        self.fc = FigureColor()
        self.fc.colorproperty = color_param

    def square(self):
        """
```

```

        """
        Класс должен переопределять метод, вычисляющий площадь фигуры.
        """
        return self.width*self.height

    def __repr__(self):
        return '{} {} цвета шириной {} и высотой {} площадью {}.'.format(
            Rectangle.get_figure_type(),
            self.fc.colorproperty,
            self.width,
            self.height,
            self.square()
        )

```

Файл circle.py

```

from lab_python_oop.figure import Figure
from lab_python_oop.color import FigureColor
import math

```

```

class Circle(Figure):
    """
    Класс «Круг» наследуется от класса «Геометрическая фигура».
    """
    FIGURE_TYPE = "Круг"

    @classmethod
    def get_figure_type(cls):
        return cls.FIGURE_TYPE

    def __init__(self, color_param, r_param):
        """
        Класс должен содержать конструктор по параметрам «радиус» и «цвет». В
        конструкторе создается объект класса «Цвет фигуры» для хранения цвета.
        """
        self.r = r_param
        self.fc = FigureColor()
        self.fc.colorproperty = color_param

    def square(self):
        """
        Класс должен переопределять метод, вычисляющий площадь фигуры.
        """
        return math.pi*(self.r**2)

    def __repr__(self):
        return '{} {} цвета радиусом {} площадью {}.'.format(
            Circle.get_figure_type(),
            self.fc.colorproperty,
            self.r,
            self.square()
        )

```

Файл square.py

```

from lab_python_oop.rectangle import Rectangle

```

```

class Square(Rectangle):
    """
    Класс «Квадрат» наследуется от класса «Прямоугольник».
    """
    FIGURE_TYPE = "Квадрат"

```

```

@classmethod
def get_figure_type(cls):
    return cls.FIGURE_TYPE

def __init__(self, color_param, side_param):
    """
    Класс должен содержать конструктор по параметрам «сторона» и «цвет».
    """
    self.side = side_param
    super().__init__(color_param, self.side, self.side)

def __repr__(self):
    return '{} {} цвета со стороной {} площадью {}'.format(
        Square.get_figure_type(),
        self.fc.colorproperty,
        self.side,
        self.square()
    )

```

Файл main.py

```

from rich.console import Console

from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square

def main():
    r = Rectangle("синего", 13, 13)
    c = Circle("зеленого", 13)
    s = Square("красного", 13)
    print(r)
    print(c)
    print(s)

if __name__ == "__main__":
    main()

console = Console()
console.print("Привет из виртуального окружения!", style="bold magenta")

```

Файл test_geometry.py

```

import unittest
from lab_python_oop.rectangle import Rectangle
from lab_python_oop.square import Square
from lab_python_oop.color import FigureColor

class TestGeometry(unittest.TestCase):

    def test_rectangle_creation(self):
        # Создание прямоугольника
        rectangle = Rectangle("красного", 5, 10)
        self.assertEqual(rectangle.width, 5)
        self.assertEqual(rectangle.height, 10)
        self.assertEqual(rectangle.fc.colorproperty, "красного")
        self.assertEqual(rectangle.square(), 50)

    def test_square_creation(self):
        # Создание квадрата
        square = Square("синего", 5)

```

```

self.assertEqual(square.side, 5)
self.assertEqual(square.fc.colorproperty, "синего")
self.assertEqual(square.square(), 25)

def test_figure_repr(self):
    # Проверка метода __repr__ для прямоугольника
    rectangle = Rectangle("зеленого", 3, 6)
    expected_repr = "Прямоугольник зеленого цвета шириной 3 и высотой 6
площадью 18."
    self.assertEqual(rectangle.__repr__(), expected_repr)
    # Проверка метода __repr__ для квадрата
    square = Square("красного", 4)
    expected_repr = "Квадрат красного цвета со стороной 4 площадью 16."
    self.assertEqual(square.__repr__(), expected_repr)

def test_color_setting(self):
    # Проверка установки цвета через FigureColor
    fc = FigureColor()
    fc.colorproperty = "оранжевого"
    self.assertEqual(fc.colorproperty, "оранжевого")

def test_square_sq(self):
    # Проверка вычисления площади квадрата
    square = Square("фиолетового", 6)
    self.assertEqual(square.square(), 36)

def test_rectangle_sq(self):
    # Проверка вычисления площади прямоугольника
    rectangle = Rectangle("голубого", 4, 8)
    self.assertEqual(rectangle.square(), 32)

if __name__ == '__main__':
    unittest.main()

```

Результат выполнения программы:
Файл main.py

```
Прямоугольник синего цвета шириной 13 и высотой 13 площадью 169.  
Круг зеленого цвета радиусом 13 площадью 530.929158456675.  
Квадрат красного цвета со стороной 13 площадью 169.  
Привет из виртуального окружения!
```

Файл test_geometry.py

```
.....  
-----  
Ran 6 tests in 0.001s  
  
OK
```